P.I. Redes / Oper

Proyecto: **Etapa #1**

Equipo: Los Padawans

Juan González Márquez - C4F588
Jeferson Marín Porras - C24549
Ariadna Ardila Gutiérrez - C4C646
Juan Sebastán Loazia Ospina B74200

Protocolo

Objetivo

Permitir a los clientes:

- Consultar figuras disponibles
- Obtener piezas de una figura específica

Arquitectura

Modelo Cliente – Intermediario – Servidor

- Cliente: realiza solicitudes y muestra resultados
- Intermediario: coordina la comunicación (núcleo del sistema)
- Servidor: almacena y gestiona la información

Flujo básico:

Cliente > Intermediario > Servidor > Intermediario > Cliente

Protocolo

- Route table
- Message format
- Message types

Tabla de rutas

- Relaciona figuras con servidores
- Permite al intermediario decidir a dónde enviar solicitudes
- Si no existe la figura > FIGURE_NOT_FOUND

Formato de mensajes

Campos principales:

- tipo, id, figura, figuras, segmento, piezas

Tipos de mensajes:

- Cliente → Intermediario: LIST_FIGURES, GET_FIGURE
- Intermediario → Servidor: ASK_FIGURE
- Servidor → Intermediario: FIGURE_FOUND, FIGURE_NOT_FOUND
- Intermediario → Cliente: FIGURES_LIST, RETURN_FIGURE, FIGURE_NOT_FOUND

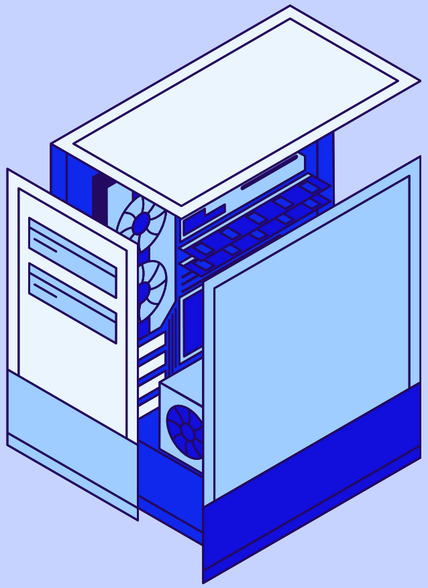
Protocolo

Operaciones principales:

- Listar figuras:
  - Se resuelve en el intermediario
- Consultar figura:
  - Se busca en tabla de rutas
  - Se consulta al servidor correspondiente

Respuestas:

- Lista de figuras (FIGURES_LIST)
- Información de figura (RETURN_FIGURE)
- Error (FIGURE_NOT_FOUND)



Otros aspectos:

- Logs: registro de eventos y errores
- Gestión de servidores:
  - Actualización automática con consultas ALL
- Manejo de errores:
  - Parámetros inválidos → valores por defecto
  - Fallos en servidores → se intenta con otros

Bitácora (**Logs**)

Se generan registros asociados a los eventos de comunicación:

- Recepción de solicitudes: LIST_FIGURES
- Generación de respuestas: PARTS_LIST_SENT
- Ocurrencia de errores: FIGURE_NOT_FOUND
- Enrutamiento de mensajes

Facilitar el monitoreo de comportamiento y apoyar procesos de depuración.

(Fecha - Hora) [Tipo] - parámetro

File System

Nuestro filesystem permite gestionar información estructurada de figuras LEGO y sus componentes, funcionando como el núcleo de almacenamiento y consulta.

La información se almacena en un archivo físico en formato JSON binario, organizado en bloques de tamaño fijo de 256 bits. Cada bloque representa una unidad de almacenamiento y contiene tanto datos como etiquetas de control que permiten interpretar su contenido.
Cada bloque de 256 bits incluye:

- Identificador del bloque.
- Tipo de contenido (figura, segmento, pieza, metadata).
- Datos útiles (nombre, cantidades, fecha de creación).
- Referencias a otros bloques por punteros lógicos.

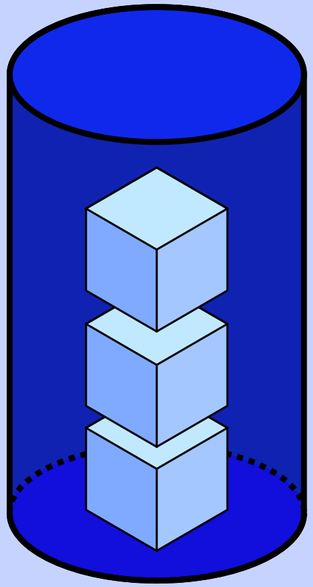


Figura **Escogida**

Death Star II

289 piezas

6441773

Identificador por pieza numerico de 7 digitos.

Descubre la increíble figura virtual de la Death Star II en mi proyecto de redes y sistemas: un modelo icónico ahora convertido en datos listos para explorarse y transmitirse. Consulta, solicita y gestiona esta figura como si fuera real, pero con el poder de los sistemas distribuidos. ¡Lleva tu experiencia LEGO al siguiente nivel, directamente desde la red!

LEGOS se venden por separado

Simulación

Cliente

Único punto de interacción con el usuario. Pide al intermediario la lista de figuras LEGOS, o las partes que contiene cualquiera de las figuras listadas.

Death Star II

Intermediario

Recibe e interpreta las peticiones del cliente para el servidor. Es un punto medio de control entre le cliente y el servidor; maneja un cache que mantiene actualizada la lista de servidores disponibles, así como las figuras que contienen.

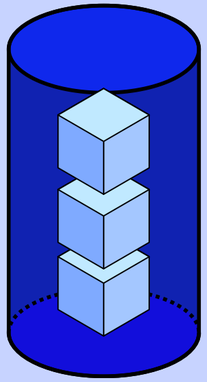
Servidores disponibles

Legos disponibles

Servidor

Centro de almacenamiento de datos donde se graba en memoria toda la informacion existente respecto a las figuras, sus segmentos, partes y metadata. Es el único que tiene acceso al filesystem y se encarga de procesar y resolver las peticiones que recibe a travez del intermediario.

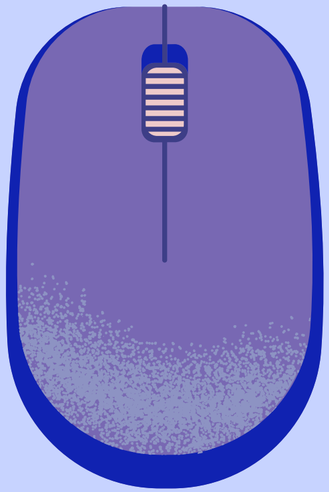
Server 1



Cliente

Está diseñado para realizar solicitudes de LEGOS según las necesidades del usuario. Y presentar la información recibida en un formato amistoso a travéz de prompts.

os.ecci.ucr.ac.cr



- Polimorfismo con VSocket
- Abstracción de Protocolo: HttpRequest y HttpResponse.
- Flujo de Operación Principal
  a. Solicitud de figuras (LIST_FIGURES).
  b. Entrada del usuario
  c. Construcción de la petición: Llama a GET_FIGURE para crear la URL.
  d. Recepción y limpieza del cuerpo del html.
  e. Despliegue de la información.

HTTP

Protocolo de comunicación cliente-servidor | Permite solicitar y recibir recursos (HTML, datos) | Usa puerto 80 (no seguro)

Solicitud

El cliente construye la perición con:

- Método: GET
- Ruta (path)
- Host

Se genera usando:

- HttpRequest::toString()

Respuesta

La respuesta del servidor incluye:

- Status line (ej: 200 OK)
- Headers
- Body (contenido HTML)

Se procesa con:

- HttpResponse::parse()
- Los datos se envían en texto plano

HTTPS / SSL

HTTP + capa de seguridad (SSL/TLS)

- Usa puerto 443
- Protege la comunicación (cifrado)
- Se utiliza SSLSocket en lugar de Socket

Proceso:

- Conexión inicial → TryToConnect()
- Asociación SSL → SSL_set_fd()
- Inicio de cifrado → SSL_connect()

Envío y recepción:

- SSL_write() → envía datos cifrados
- SSL_read() → recibe datos cifrados

Beneficios:

- Confidencialidad
- Integridad
- Autenticación

Casos **de prueba**

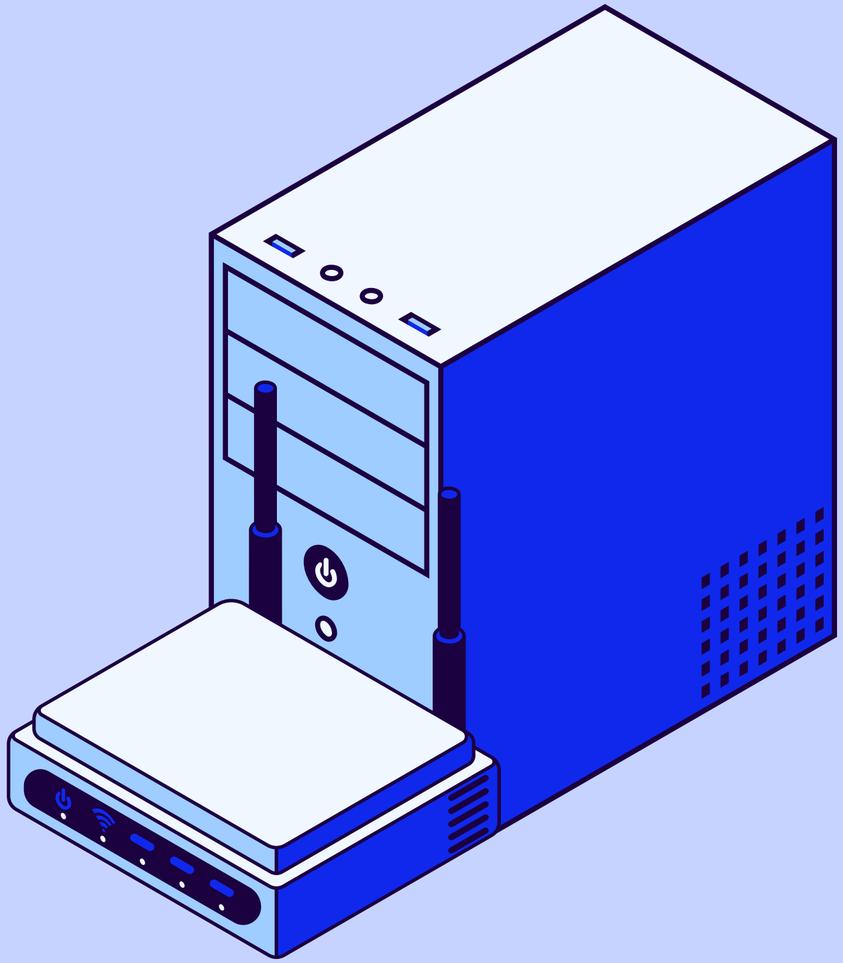
Caso de prueba	Entrada / Condición	Resultado esperado	Estado
Selección válida de figura y primera mitad	Elegir una figura válida y opción 1	Se envía request correcto y se muestra la primera mitad de la figura	Manejado correctamente
Selección válida de figura y segunda mitad	Elegir una figura válida y opción 2	Se envía request correcto y se muestra la segunda mitad de la figura	Manejado correctamente
Varias solicitudes seguidas	Hacer varias consultas consecutivas en una misma ejecución	El programa responde correctamente en todas las solicitudes	Manejado correctamente
Respuesta corresponde a la figura pedida	Pedir una figura específica	La respuesta mostrada coincide con la figura solicitada	Manejado correctamente
Respuesta corresponde a la mitad pedida	Pedir mitad 1 o 2	El contenido mostrado corresponde exactamente a la mitad seleccionada	Manejado correctamente
Selección válida de figura y primera mitad	Elegir una figura válida y opción 1	Se envía request correcto y se muestra la primera mitad de la figura	Manejado correctamente
Modo HTTP funciona	Ejecutar sin parámetro SSL	El cliente conecta por puerto 80 y recibe respuesta válida	Manejado correctamente
Modo HTTPS funciona	Ejecutar con parámetro SSL	El cliente conecta por puerto 443 y recibe respuesta válida	Manejado correctamente

Casos **de prueba**

Caso de prueba	Entrada / Condición	Resultado esperado	Estado
Repetir consulta	Responder s al mensaje “¿Desea solicitar otra figura?”	Se realiza una nueva consulta sin cerrar el programa	Manejado correctamente
Finalizar consulta	Responder n al mensaje final	El programa termina correctamente	Manejado correctamente
Listado de figuras cargado correctamente	Iniciar cliente con lista vacía	El sistema obtiene o carga el listado de figuras disponibles	Manejado correctamente
Entrada vacía en selección de figura	Presionar Enter sin escribir nada	Se muestra mensaje de “Entrada vacía” y vuelve a pedir la opción	Manejado correctamente
Entrada vacía en selección de mitad	Presionar Enter sin escribir nada	Se muestra mensaje de “Entrada vacía” y vuelve a pedir la opción	Manejado correctamente
Texto en vez de número en figura o texto en vez de número de mitad	Escribir algo como la letra ‘h’	Se muestra “Entrada inválida” y se vuelve a pedir la opción	Manejado correctamente
Número fuera de rango en figura y mitad	Escribir 0, -1 o un número mayor al listado	Se muestra “Opción inválida” y se vuelve a pedir la opción	Manejado correctamente
Caracteres especiales en figura	Escribir $	Se detecta como inválido y no se envía request	Manejado correctamente

Casos **de prueba**

Caso de prueba	Entrada / Condición	Resultado esperado	Estado
Espacios en blanco como entrada	Escribir solo espacios	Se rechaza la entrada y vuelve a solicitarla	Manejado correctamente
Conexión imposible cliente-servidor	Servidor apagado o IP incorrecta	El cliente reporta fallo de conexión	Manejado correctamente
Canal interrumpido durante lectura	Cortar conexión mientras llega la respuesta	El cliente maneja la lectura parcial sin comportarse de forma inesperada, espera a que vuelva la conexión	Manejado correctamente
Muchas solicitudes consecutivas	Realizar 10 o más consultas seguidas	El programa mantiene estabilidad y responde en todas	Manejado correctamente
Solicitudes alternando figuras	Pedir distintas figuras una tras otra	Cada respuesta corresponde a la solicitud actual, no a una anterior	Manejado correctamente
Cierre limpio después de varias consultas	Hacer varias consultas y luego responder n	El programa termina sin errores	Manejado correctamente
Letra distinta de s/n al continuar	Responder h	El programa interpreta que no continúa y finaliza	Manejado correctamente



Switch
Packet tracer

Etapa 1 - Equipo 3

Muchas **gracias**